

WORKSHOP · 2026

2 DAYS · 4 HRS
PRODUCT & UX DESIGNERS

Agentic Harness Engineering

Reverse-engineer & build the scaffolding that makes AI useful.

2 DAYS · 2 HRS EACH · 5 HANDS-ON BREAKOUTS

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Welcome to Agentic Harness Engineering for Product & UX Designers. This is a 2-day, 2-hour-per-day workshop. Quick show of hands — who's been using RooCode? Copilot? Both? Great. You already touch the harness every day — you just call it 'rules' or 'instructions'. Today and tomorrow we name what you're already doing, give it a real anatomy, and you build one from scratch by reverse-engineering an AI app you actually use. By the end of Day 2 you'll have a runnable harness folder you can drop into any project.

WELCOME

Workshop overview & takeaways

Two days of vocabulary + hands-on files. By the end you reverse-engineer a real productized harness, then assemble your own harness folder using the same anatomy.

TAKEAWAY 01

Speak harness anatomy fluently.

System prompt, skills, tools, rules, hooks, knowledge, memory — what each does, where it lives on disk, and how product teams ship them.

TAKEAWAY 02

Reverse-engineer before you prompt.

Trace Harvey, Claude for Financial Services, or Granola through the same six slots; carry that lens into every AI surface you design.

TAKEAWAY 03

Leave with a real folder.

Five breakouts → five artifacts in your template project (prompt, skill, tool spec, hooks/rules wiring, knowledge + memory notes).

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Workshop overview & takeaways (slide 1b). Give ~30 seconds per card: fluent anatomy vocabulary, reverse-engineer before raw prompting, leave with artifacts in the template repo. Tie each takeaway to today's five breakouts without opening the facilitator manual.

CONTEXT

Why this workshop matters

01 · PARADIGM SHIFT

From static wireframes to dynamic orchestration rules.

Designers now shape how agents coordinate — not just how screens look.

02 · AGENT = MODEL + HARNESS

The model is the intelligence. The harness makes it useful.

Designers have massive influence over harness decisions.

03 · DESIGN IS ENGINEERING

OpenAI built a million-line product where humans steer and agents execute.

The discipline moved to the scaffolding.

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

The role of design is shifting from crafting interfaces to orchestrating intelligent systems. CARD 1 — Paradigm shift: from static wireframes to dynamic orchestration rules. CARD 2 — Agent = Model + Harness. The model is the brain. The harness is the body, the workshop, and the rules of engagement. Designers shape the harness. CARD 3 — Design IS engineering. OpenAI shipped a million-line product where humans steered and Codex executed. The discipline is now in the scaffolding.

AGENDA		
Two days, one productized app		
DAY 1 - 12h MINBuild the harness		
8:00	Foundations	Saddle - Agent + Model + Harness - Rings - What models skip alone.12
8:12	Anatomy overview	Six components - files & editors - Harvey + Claude PS + Granola demos - choose breakout app (mains).10
8:22	Prompt framing - Breakouts	Meta vs template prompts - skill chains preview - five hands-on labs (prompt → skills → tools → hooks/rules → knowledge/memory).8
8:38	Breakout 1 - System prompt	Identify + anatomy teach → write yours in the template repo (companion page).12
8:42	Breakout 2 - Skills	Triggers & SKILL and anatomy → one skill (use packaged aka111+crux as if helpful).15
8:57	Breakout 3 - Tools	JSON + scripts/MCP teach → one tool definition + description craft.18
1:15	Rules + Hooks teach	Soft vs hard rules - pre/post hook model (paired).8
1:23	Breakout 4 - Hooks & rules	Numbered rules + para-tool+aaa.als that enforces the "never."12
1:35	Breakout 5 - Knowledge & memory	One stable knowLedge / file + session vs persistent memory plan.12
1:47	Skill chains in production	How prompts compose with tools, hooks, rules, and loops for long runs.8
1:55	Show & tell	Each group: artifact folder + one open question for Day 2.5
AGENTIC HARNESS DESIGNING WORKSHOP		
1:57	Homework	Join UX notes for your harness UI and read Copilot / RooCode docs before Day 2.9

SPEAKER NOTES

Day 1 you build the harness for your product. Day 2 you write a short UX brief, then in one VS Code app project use GitHub Copilot or RooCode to refine the harness and build the UI before wiring agent, harness, and UX together. — Two days, one product. Day 1 you build the harness for your chosen app — six components, taught and worked in focused breakouts. Day 2 changes shape: you clarify the frontend in prose or light sketches, generate the code with Copilot or RooCode, then wire the agent + harness + frontend UX together into a productized demo. I want you 60% hands-on, 40% with me. Let's go.

SETUP

Your environment & ground rules

STACK YOU HAVE

- **VS Code** — your editor
- **RooCode** or **GitHub Copilot** — pick one extension; either works as the agent surface for building your harness.
- A short **UX brief** (bullets or doc) for the screens you want
- **Optional: MCP servers** (e.g. GitHub)

WHAT YOU'LL BUILD

- **The harness** — markdown, JSON, scripts
- **A frontend prototype** — generated in your VS Code project with Copilot or RooCode from that brief.
- **An eval set** — 3–5 captured tasks the harness should handle
- **A demo script** — one task you'll run live

Before Day 1 breakouts: Install VS Code, enable your AI extension (Copilot or RooCode), clone or unzip the workshop harness template (`exercises/sample-harness-uxd-specs` or `harness-templates/`), and open it as the workspace. Download the [starter pack \(skills + slash commands\)](#). Companion instructions for each breakout live under `docs/workshop-breakouts/`.

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

We're not training models or running servers today. You'll author the harness and a frontend prototype, then let your editor's AI play the agent. GROUND RULES — read this slide carefully because it shapes everything. CRITICAL CONSTRAINT: API keys are only available INSIDE the RooCode and Copilot extensions today. You cannot build a standalone agent that calls a model directly. The extensions already have the key — you don't. So we don't pretend to ship a real backend. Instead, we work with what you DO have: VS Code + RooCode + Copilot. THE EDITOR IS THE RUNTIME. You author two things: (1) the harness — markdown, JSON, scripts in a folder — and (2) a frontend prototype built with Copilot or RooCode from your UX brief. Then you load both into VS Code and let the extension's AI play the harnessed agent. Your CLAUDE.md is its system prompt. Your hooks fire on real tool calls. Your prototype is what a user clicks. This is exactly what Day 2's wire-up exercise asks you to do. One sentence to remember: 'harness + frontend = product; the editor's AI is the runtime.'

01

SECTION · FOUNDATIONS

Foundations

What "harness" means here—and why it matters to design work.

15 MINUTES

The harness sits between the user and the model: prompts, tools, guardrails, and workflow that turn raw capability into something steerable. **This section is vocabulary and framing only. Files and product demos come after we introduce the harness anatomy.**

Saddle metaphor

Core formula

Rings & gaps

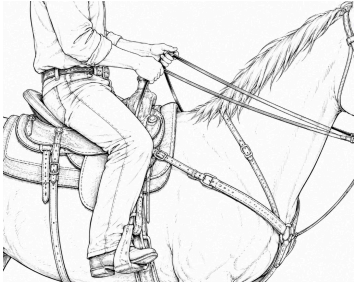
ALBERTO BARREDA ENGINEERING WORKSHOP

SPEAKER NOTES

[Pause.] Foundations: name what sits between user and model—then we'll mirror it into files and products.

FOUNDATIONS · THE METAPHOR

What is a harness?



AGENTIC HARNESS ENGINEERING WORKSHOP

THE HORSE

The model has the power.

Raw intelligence — fast, capable, untrained on your task.
Powerful enough to throw a rider.

THE SADDLE

The harness makes it rideable.

A seat to sit on, stirrups to steer, a girth to hold the work in place. Not the power — the leverage.

THE RIDER

The user gives direction.

Without the saddle, the user is on the ground. With it, they go where they meant to go.

SPEAKER NOTES

A model is a brilliant mind with no body. The harness is the tack that lets it carry weight, take direction, and do useful work. — Plant the metaphor. The model is the horse — fast, strong, but it has no reins, no seat, no destination. The saddle isn't the power; it's the leverage. Three pieces map across: model = horse (raw capability), harness = saddle (tools, rules, memory, hooks — the tack that makes the horse rideable), user = rider (gives direction). This is the working metaphor for the rest of Day 1: every component you build is a piece of saddle. If a model alone could do it, you wouldn't be here.

FOUNDATIONS

The core formula

AGENT = MODEL + HARNESS

Agent

The emergent behavior

Goal-directed, tool-using, self-correcting entity the user experiences.

Model

The intelligence

Takes in text and images, outputs text. That's it.

Harness

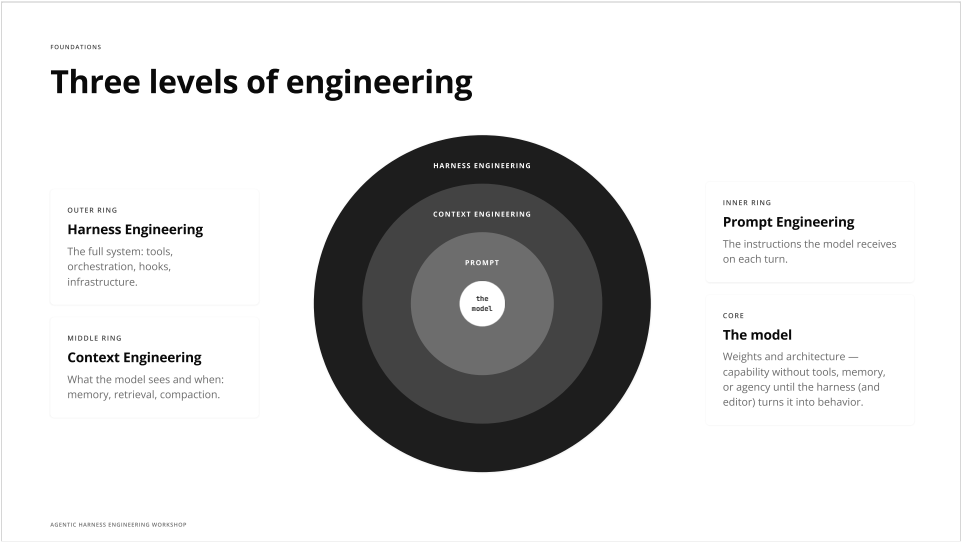
Everything else

Code, config, tools, constraints that make the model useful.

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Most important idea in the workshop. Agent = Model + Harness. Vivek Trivedy at LangChain: 'If you're not the model, you're the harness.' The AGENT is what the user experiences. The MODEL takes text and images and emits text — that's all. The HARNESS is everything else: code, config, files, tools, constraints. Designers don't train models — they absolutely design harnesses.



SPEAKER NOTES

Three concentric levels. Prompt — what the model sees this turn. Context — what the model sees and when across turns. Harness — the entire system. We touch all three but harness is where product decisions live.

FOUNDATIONS

What models can't do out of the box

Remember past sessions

→

Memory & persistent storage

Execute code or actions

→

Tool execution & sandboxes

Access the internet

→

Web search, API integrations

Coordinate with others

→

Orchestration & handoffs

Verify their own output

→

Self-evaluation loops

Enforce rules reliably

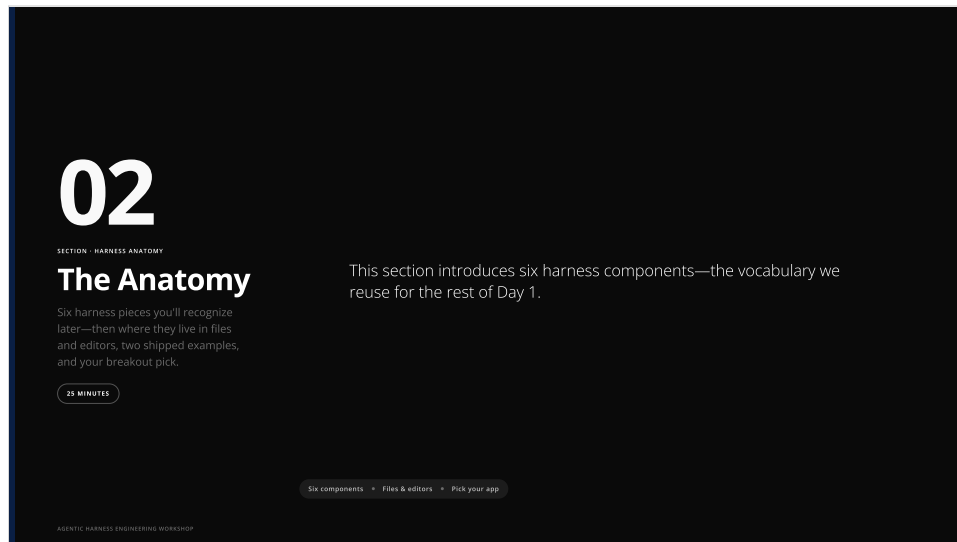
→

Hooks & middleware

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Every limitation is solved by a harness feature. This is where your design decisions live. — Six things models can't do alone. Each one solved by a harness feature. Memory · tool execution · web · coordination · self-verification · enforcement. Every row is a design decision your team will make.



SPEAKER NOTES

SECTION divider · The Anatomy

[Pause.]

We're opening section two.

Foundations ended with what models can't do alone—and how harness pieces answer each gap.

Now everyone gets the same map before we go deep: shared vocabulary first, then the marathon teach-and-dive runs.

I'll walk through what's coming. Say this slowly—the slide is the cheat sheet.

First: the six-slot overview on one screen. Orient the room; don't ask anyone to memorize yet.

Next: pairwise slides and the leverage map—we widen the picture.

Then: the file-tree slide—where those slots land as real folders.

Then: two shipped demos—Harvey and Claude for Financial Services. Same six labels; different products.

Last in this block: you pick one breakout app from the matrix—Harvey, Claude Financial Services, or Granola.

Close with this line: the slow, component-by-component pass starts in section three, right after you've committed to that pick.

ANATOMY

Six core harness components

01



System prompt

Meta vs template framing · identity & structure → write yours.

02



Rules

Strong vs weak · write three numbered rules.

03



Skills

What / anatomy / example → design one skill.

04



Tools

Functions · scripts · MCP · descriptions → define one.

05



Hooks

Pre/post gate model → wire one pre-tool guard.

06



Knowledge base

What belongs in vs out → capture one domain file.

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Six cards mirror Day 1 agenda breakouts after anatomy—not abstract buckets: system prompt, rules, skills, tools, hooks, knowledge base. Prompt-structure framing folds into card 01; memory is teach-only later (see agenda). Say explicitly each chip maps to a breakout you'll run.

ANATOMY

System prompt & tools

01 System Prompt & Instructions

The persona, rules, and constraints you give the model.

DESIGN DECISIONS

Tone & personality of the agent

What it should / shouldn't do

AGENTS.md / .github/copilot-instructions.md

Prompt files as progressive disclosure

DESIGNER LEVERAGE

High

02 Tools, Skills & MCPs

The actions the agent can take in the world.

DESIGN DECISIONS

Which tools to expose (fewer = better)

Tool descriptions (UX copy for agents)

MCP server connections

Permission boundaries & confirmations

DESIGNER LEVERAGE

Very High

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

LEFT: System prompt = persona, rules, constraints. AGENTS.md and .github/copilot-instructions.md tell the agent how to behave. Prompt files are progressive disclosure. RIGHT: Tools are the actions in the world. Insight: too many MCP servers make agents dumb. Tool descriptions are UX copy for the model — fewer, sharper tools beat a kitchen sink.

ANATOMY

Infrastructure & orchestration

03 Infrastructure & Sandbox

The workspace the agent operates in.

DESIGN DECISIONS

- Filesystem as the foundational primitive
- Browser access for web tasks
- Sandboxed execution environments
- What data is in-context vs on-disk

DESIGNER LEVERAGE

MEDIUM

04 Orchestration Logic

How agents coordinate & delegate.

DESIGN DECISIONS

- When to spawn subagents
- Agent handoff protocols
- Model routing (fast vs smart)
- Sprint contracts between agents

DESIGNER LEVERAGE

HIGH

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

LEFT: Infrastructure — filesystem is the foundational primitive. Decide what's in-context vs on-disk. RIGHT: Orchestration — when to spawn a subagent, when to keep work in the main thread. These are product decisions, not engineering decisions.

ANATOMY

Hooks & feedback loops

05 Hooks & Middleware

Deterministic logic around the agent.

Key insight. When something should happen every time, don't rely on the model to remember — enforce it with code.

EXAMPLES

• Auto-lint after every code edit

• Safety checks before tool execution

• Context compaction at thresholds

06 Feedback Loops

How the harness improves over time.

THE IMPROVEMENT CYCLE

• Traces — understand failure modes

• Evals — the "training data" for the harness

• Self-verification — agent checks its own work

• Hill-climbing — iterative harness optimization

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

LEFT: Hooks. When something should happen EVERY time, enforce it with code, not with hope. RIGHT: Feedback loops. Evals are the training data for the harness. LangChain calls this hill-climbing — change one thing, measure, keep or revert.

ANATOMY

Where designers have the most leverage

 Most agent failures aren't intelligence failures. They are input failures.

**Tool descriptions**
The "UX copy" for AI agents — bad descriptions = wrong tool choices.

**State & progress visibility**
Users need to understand what agents are doing and why.

**Agent coordination rules**
Which agents work together, handoffs, escalation paths.

**Permission & trust UX**
When to ask, when to act, what to explain.

**Error recovery flows**
What happens when the agent fails?

**Input data architecture**
Agent Time: Past (prefs), Present (state), Future (adaptation).

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Where designer leverage is highest: tool descriptions, permission UX, state visibility, error recovery, agent coordination, input architecture. Most agent failures are input failures, not intelligence failures.

ANATOMY

The harness is mostly files

Markdown for instructions. JSON for config. Shell scripts for hooks. The agent reads them every turn.

```
my-app-harness/
├── AGENTS.md # project context, conventions, rules - read by Copilot
├── .github/
│   ├── copilot-instructions.md # Copilot repo-wide instructions
│   ├── instructions/
│   │   ├── frontend.instructions.md # path-scoped instructions (applyTo: glob)
│   │   └── backend.instructions.md
│   ├── prompts/
│   │   ├── plan.prompt.md # reusable /plan prompt
│   │   └── ship.prompt.md # reusable /ship prompt
│   ├── agents/
│   │   ├── researcher.md # custom Copilot agent
│   │   └── reviewer.md
│   └── hooks/
│       ├── pre-tool-use.sh # gate dangerous calls (cloud agent)
│       └── post-tool-use.sh # auto-format, lint, log
├── .roo/
│   ├── rules/01-design.md # RooCode workspace rules
│   ├── rules-(mode)/
│   └── mcp.json # external tool servers (MCP)
```

CLAUDE.md / AGENTS.md

The system prompt you actually own
Highest leverage. Keep under 200 lines. Survives compaction.

.claude/settings.json

Permissions & hook bindings
Allow / deny tools, env vars, which scripts run on which events.

.claude/skills/*.md

Reusable, discoverable prompts
Loaded only when relevant — progressive disclosure.

>_ .claude/hooks/*.sh

Deterministic guardrails
Every time the model touches X, this script runs. No "the model might forget."

mcp.json

External tool servers
GitHub, browser, Confluence (examples — setup required)

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

NEW SLIDE — Anatomy in files. The six components live as concrete files: AGENTS.md, .github/copilot-instructions.md, .github/instructions/, .github/prompts/, .github/agents/, .github/hooks/, mcp.json, plus the RooCode/Cline shape under .roo/rules/. Walk through the tree. Highest-leverage files: AGENTS.md (the system prompt you own), tool descriptions inside prompt files (UX copy for the model), and hooks (deterministic enforcement). Tell the room: 'You're going to author every one of these in Day 1's breakouts.'

ANATOMY · SHIPPED HARNESS

When a harness becomes a product

Harvey is AI designed for legal and professional services. Advance your expertise on a secure platform that lets you focus on high-value work.

Client notes

I represent Acme Corp in the attached lawsuit. Research the strongest pieces of evidence in my client's favor and draft an email.

FileSourcesImprove

Undo

Copy

Redo

Share

PERSONA & RULES

Domain-specific assistant

Purpose-built for legal — privilege, citations, jurisdictional grounding baked into the system prompt.
↳ SYSTEM PROMPT

MEMORY & RETRIEVAL

Source-grounded answers

Vault is the knowledge base; every output cited back to documents the firm trusts.
↳ KNOWLEDGE BASE • RAG

HOOKS & GOVERNANCE

Privacy & security gates

Privilege protection, audit trails, deterministic guardrails the model can't bypass.
↳ HOOKS • MIDDLEWARE

TOOLS

Vault • Knowledge • Workflow Agents

Document store, legal/regulatory/tax retrieval, and pre-built agents for litigation, transactional, in-house.
↳ TOOLS • SKILLS

ORCHESTRATION

Specialized agents end-to-end

Distinct workflow agents per practice area; firms build their own within bounds.
↳ SUBAGENTS • HANDOFFS

INTEGRATIONS

Ecosystem layer

Plugs into the firm's existing workflows — DMS, mobile capture, collaboration spaces.
↳ MCP-STYLE CONNECTORS

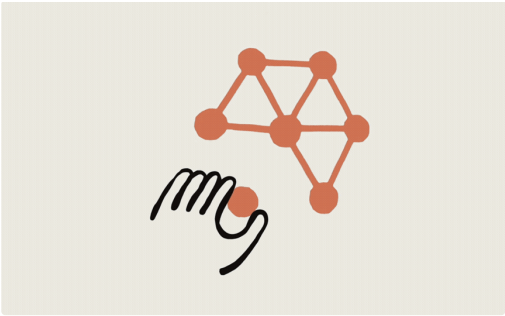
AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Harvey AFTER the scaffolding on purpose — each bullet NAMES a slot learners just saw on the grid + filesystem + editor columns. Tie line: polished vendor harness vs bare repo they'll assemble for their OWN app.

ANATOMY · SHIPPED HARNESS · FINANCIAL SERVICES

Claude for Financial Services



PERSONA & RULES

Risk-aware, verification-first stance

Built for banking, insurance, asset management, and fintech — safety and source-attributed outputs framed for committees and second-line review.

~ SYSTEM PROMPT

MEMORY & RETRIEVAL

Traceable numbers

Every figure tied back to data the institution trusts — not just fluent prose, but auditable lineage from signal to narrative.

~ KNOWLEDGE BASE + RAG

HOOKS & GOVERNANCE

Enterprise assurance

Controls your risk function expects — e.g. SOC 2 and FedRAMP posture — with deterministic gates before capital markets or client-facing actions.

~ HOOKS + MIDDLEWARE

TOOLS

Excel · PowerPoint · finance agent templates

Native in the spreadsheets and decks analysts already use; agent templates and add-ins compress modeling, diligence, and reporting workflows.

~ TOOLS + SKILLS

ORCHESTRATION

End-to-end financial agents

Multi-step flows such as AML investigations, credit decisioning, fraud prevention, and portfolio ops — handoffs across specialized agents.

~ SUBAGENTS + HANDOFFS

INTEGRATIONS

Market & identity data partners

Pre-built connectivity to LSEG, FactSet, S&P Global, Morningstar, Moody's, Dun & Bradstreet, and more — verifiable data where decisions happen.

~ MCP-STYLE CONNECTORS

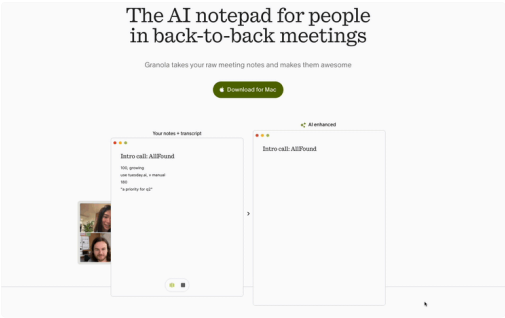
AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Claude Financial Services pairing — SAME six-bullet mapping in regulated finance (verification posture, workbook-native tools + agent templates, traceable numeric lineage, AML-style multi-agent stories, SOC 2/FedRAMP cues, MCP-like data integrations). Explicit ask: naming pattern should feel boringly familiar now.

ANATOMY · SHIPPED HARNESS · MEETINGS

Granola — meeting-native harness



AGENTIC HARNESS ENGINEERING WORKSHOP

PERSONA & RULES

Neutral chronicler

Tone calibrated for notes stakeholders share
— avoids hallucinating commitments;
Surfaces uncertainty explicitly.
↳ SYSTEM PROMPT + RULES

SKILLS

Templates per meeting type

Standups vs sales calls vs interviews →
different summarization rubrics loaded on
demand.
↳ SKILLS

HOOKS & GOVERNANCE

Consent & retention

Recording gates, delete/export paths, org
policies enforced before audio touches cloud
speech APIs.
↳ HOOKS + MIDDLEWARE

TOOLS

Calendar · CRM · docs

Writes structured recap blocks and action
items into the systems teams already use.
↳ TOOLS + INTEGRATIONS

MEMORY & RETRIEVAL

Episode memory

Links today's decisions to prior meetings
without stuffing everything into one mega
prompt.
↳ KNOWLEDGE BASE + RECALL

ORCHESTRATION

Realtime → async cleanup

Streaming capture followed by chained
refinement passes once the meeting ends.
↳ SKILL CHAINS + LOOPS

SPEAKER NOTES

Granola harness slide (08d). Meeting-native productivity — emphasize how a minimal UI still maps to all six slots, especially orchestration / skill chains. Optional energy line: 'Notes without stopping the meeting' is a harness problem, not a prettier textarea.

DAY 1 - AFTER THE SHIPPED EXAMPLES

Pick one app to reverse-engineer

A

LEGAL · ENTERPRISE

Harvey

Legal AI platform for law firms. Research, drafting, document analysis, conversational queries over a firm's knowledge.

Why pick: rich agentic harness — multi-step research, document tools, strict citation & privilege rules.

B

FINANCIAL SERVICES · REGULATED

Claude Financial Services

Vertical Claude for banking, insurance, asset management, and fintech — verification-first outputs, native in Excel and PowerPoint, agent templates for modeling and reporting workflows.

Why pick: you already mapped this harness on the shipped demo — now reverse-engineer the full product story, tools, and governance layer, not just the landing page.

C

PRODUCTIVITY · MEETINGS

Granola

AI notepad for back-to-back meetings. Listens, drafts notes, extracts actions, syncs to your tools.

Why pick: smallest surface area, sharpest job-to-be-done — easy to map all 6 components in 25 minutes.

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

ON-SCREEN (was slide lede — read aloud while they study the matrix): You just traced the same six slots through Harvey (legal platform) and Claude for Financial Services. Now commit to one product you'll unpack: three options on the matrix below — same anatomy exercise, scoped to software your group knows.

ADDITIONAL FACILITATOR NOTES: You can also say we walked Granola on the previous slide using the same six labels. Commit to one column for reverse-engineering — then implement each breakout inside your downloaded template harness project (sample-harness-uxd-specs / harness-templates), swapping persona and domain language to match that product. PICK slide timing: ~5 minutes; diversify groups across A/B/C so they don't all pile on Harvey. Energy line option: 'Pick the harness you'd hate to embarrass users with.'

03

DAY 1 · COMPONENT DEEP DIVE

One component at a time

Five sequenced breakouts map to real files: system prompt → skills → tools → hooks/rules → knowledge/memory. Meta-vs-template and skill-chain slides close the loop after you've touched every lever.

90 MINUTES

You've picked a product to reverse-engineer. Each slot on the anatomy now gets a teaching pattern plus a breakout wired to **that** app. **Depth over breadth — leave with a real folder, not slides.**

90 minutes · Hands-on breakouts · One harness folder

ALBERT HARNES ENGINEERING WORKSHOP

SPEAKER NOTES

SECTION divider · Component deep dive. Anatomy orient + demos + breakout choice are anchored — reset expectations: ninety minutes drilling each harness slot ON their chosen reverse-engineered app. Depth beats another overview.

COMPONENT 1 · SYSTEM PROMPT

The agent's identity, prepended every turn

Persistent context. Survives compaction. Sets persona, scope, and the rules the model is supposed to remember. Most expensive real estate in the harness — keep it short and load-bearing.



The non-negotiables

Identity, scope, hard rules, escalation, failure modes.

LIVES IN

- `AGENTS.md` — read by Copilot's coding agent & Cline / RooCode
- `.github/copilot-instructions.md` — Copilot repo-wide brief
- `.roo/rules/*.md` — RooCode workspace rules



What it earns you

Consistent behavior across turns and sessions.

FAILURE IF MISSING

- Agent re-asks scope every session
- Drifts off-task halfway through long work
- Forgets the rules you said in chat

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

The system prompt is the agent's identity — prepended every turn, survives compaction. Most expensive real estate in the harness, so keep it short and load-bearing. Lives in `CLAUDE.md`, `AGENTS.md`, or `.github/copilot-instructions.md` depending on your editor. Failure mode if it's missing or vague: agent has no consistent persona, drifts in tone, forgets your hard rules between turns. The room mostly already has one — they just call it 'rules' or 'instructions'. Today we name the shape.

COMPONENT 1 · SYSTEM PROMPT

Anatomy of a high-quality system prompt

Five sections, in order. Each one earns its place — if you can't say what failure mode a section prevents, cut it.

```
flowchart TD
    A["1 - Identity  
Who is this agent? Who do they serve?"] --> B["2 - How you work  
3-5 step pattern the agent follows"]
    B --> C["3 - Rules  
3-5 never / always constraints"]
    C --> D["4 - Background  
Domain context the agent always reads"]
    D --> E["5 - When things go wrong  
Specific instructions per failure mode"]
```

```
style A fill:#fff7ad,stroke:#f59e0b
style B fill:#fff6ff,stroke:#3b02f6
style C fill:#fef2f2,stroke:#ef4444
style D fill:#f0fde4,stroke:#22c55e
style E fill:#fae5ff,stroke:#a555e7
```

Now · Breakout 1 · 12 min

Write your agent's system prompt for the app you picked — identity + scope + 3 rules + failure modes. Keep under ~200 lines; mirror your template's prompts/ or AGENTS.md pattern.

Lab sheet →

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Five sections, in order. IDENTITY — who you are. SCOPE — what you do and don't do. RULES — hard predicates. TOOLS — the roster. FAILURE MODES — what to do when stuck. Each section earns its place: if you can't name the failure mode it prevents, cut it. The mermaid diagram on screen IS the template — copy it for your harness in the breakout. Don't add a sixth section unless you can defend why.

COMPONENT 2 · SKILLS

Skills: prompts loaded on demand

A skill is a specialist instruction set the agent loads **only when a trigger fires**. The base context stays small; expertise is paged in.

```
flowchart LR
    User["User input"] --> Match{"Trigger match?"}
    Match -->|yes| Load["Load skill prompt  
(skills/design-review.md)"]
    Match -->|no| Plain["Continue with base prompt"]
    Load --> Apply["Apply skill steps"]
    Apply --> Reply["Reply to user"]
    Plain --> Reply
```

Without skills, every specialist instruction lives in the base prompt — the model carries weight it doesn't need 90% of the time.

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Strong vs weak rules. Weak: 'be helpful' — the model can't enforce that, no predicate to check. Strong: 'never write outside /specs without confirmation' — clear, checkable. Soft rules go in the system prompt; the model may forget them. Hard rules go in settings.json allow/deny lists or in hooks; the harness enforces them. Walk the room: which of YOUR rules are hard, which are soft?

COMPONENT 2 · SKILLS

Anatomy of a skill file

A skill is a markdown file with frontmatter that tells the harness when to load it, plus a body that tells the model what to do once loaded.

```
---
name: design-review
description: Reviews UX changes against the design system.
  Trigger: user asks to "review", "audit",
  or "check" a design or PR. Do NOT load
  for general code review.
---
```

Design Review

- When a design review is requested, follow these steps:
1. Read the changed files in the PR.
 2. For each component reference, check knowledge/design-system-index.md.
 3. Flag mismatches as 'design_system.unknown_component'.
 4. End with a 1-sentence verdict: *ship*, *fix first*, or *needs human*.

The description: field is the most important. The model reads it to decide whether to load the skill — write it like UX copy, not docs.

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

A skill is a specialist instruction set the agent loads ONLY when triggered. Base context stays small; expertise gets paged in. This is progressive disclosure for AI. Trigger by keyword, slash command, file pattern, or user intent. The trigger is the most important field — vague triggers mean the skill never loads, then the agent falls back to its base instinct. The whole reason skills exist: keep the system prompt short.

COMPONENT 2 · SKILLS

Skill in action: a design-review pass

User asks for a review. The trigger matches. The harness loads the skill. The agent follows the skill's steps — not its base instinct.



Generic review

"Here are some thoughts on the design:

- Looks clean overall
- Maybe consider spacing
- The button could be bigger"

Soft, unfocused. No trace back to your design system. Useful only as a vibe check.



Structured pass

Component check:

- ✓ Button — matches DS index
- △ Fill — not in DS, did you mean Chip or Badge?
- ✗ NavTab — unknown component

Verdict: **fix first** (1 unknown component blocks ship)

Specific, actionable, traceable. The skill made the agent *useful*.

Now · Breakout 2 · 12 min

Design one skill — YAML frontmatter (name + trigger description) + numbered body. Optional: install `skill-creator` from the starter pack and ask the agent to draft the `SKILL.md`.

Lab sheet →

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

A skill is just a markdown file with frontmatter. Frontmatter tells the harness WHEN to load it: name, description (used for trigger matching), trigger pattern. Body tells the model WHAT to do once loaded. Treat the description like UX copy — vague descriptions cause vague triggering. Same writing discipline as a tool description.

COMPONENT 3 · TOOLS

Functions, scripts, and MCP — three ways to give the agent hands

Same idea, different transport. Pick the one that matches where the capability already lives.

**Native tool call**

A JSON-described function the model invokes directly.
Built into the agent SDK.

USE WHEN

- The capability is in-process
- You control the runtime
- Latency matters

**Bash / shell tool**

A binary the agent runs via Bash. Stdout is the return value.

USE WHEN

- The capability is already a CLI
- You want code-level enforcement
- Logs matter for debugging

**External server**

A standalone server the agent connects to over HTTP/stdio. Hosts a toolset.

USE WHEN

- The capability lives elsewhere (Linear, Github, a hosted API)
- Multiple agents share it
- Auth / quota live there

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Walk through a real flow. User says 'review my homepage'. The harness sees the trigger 'review', loads the design-review skill, agent now follows the skill's steps instead of its base instinct. Without the skill: a generic critique. With the skill: token violations, line numbers, fix suggestions — the shape you wanted. Point: skills aren't decorative, they CHANGE the agent's behavior on the matching turn.

COMPONENT 3 · TOOLS

The tool description is UX copy for the model

The model reads every tool's description on every turn before deciding what to call. Bad descriptions = wrong tool picks. Three sentences: **what it does**, **when to use it**, **when NOT to use it**.

Weak

Generic, ambiguous

```
{
  "name": "validate",
  "description": "Validates files."
}
```

Model has to guess: which files? when? based on what? Will pick this for everything that smells "validation."

Strong

Specific boundaries

```
{
  "name": "validate_spec",
  "description": "Validates a spec
against knowledge/spec-schema.md.
Use before any status transition,
before saving an in-review spec.
Do NOT use as a general read -
use Read for that."
}
```

Tells the model exactly when this tool is the right pick — and when it isn't.

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Three ways to give an agent hands. FUNCTIONS — code in the same process. SCRIPTS — anything you can run from a shell. MCP — borrow someone else's toolset over a server protocol. Same idea, different transport. Pick by where the capability already lives. Python lib you have? Function. Curl + jq pipeline? Script. Third-party ticketing API you don't want to shell-wrap? MCP. You don't have to invent the integration.

COMPONENT 3 · TOOLS

Script-as-tool: the JSON points, the script does

The JSON declares the interface. The script does the work. The agent reads stdout.

terfa tools/validate_spec.json

```
{
  "name": "validate_spec",
  "description": "...",
  "parameters": {
    "spec_id": { "type": "string" },
    "strict": { "type": "boolean" }
  },
  "implementation":
    "scripts/validate_spec.sh"
}
```

men scripts/validate_spec.sh

```
#!/usr/bin/env bash
# Validates against schema rules.
# Emits JSON to stdout.

required_fields=(id title type
status owner created updated
version)
for f in "${required_fields[@]}";
do ...
done

echo '{"ok":true,"errors":[]}'
```

Why scripts? You can read the diff, write tests, audit logs. Pure prompts can't do any of that.

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

The tool description is UX copy for the model. The model reads every tool description on every turn before deciding what to call. Bad descriptions = wrong tool picks more often than missing tools. Three sentences: WHAT it does, WHEN to use it, WHEN NOT to use it. The 'when not' line saves you most failures — it teaches the model to delegate or refuse instead of jamming the wrong tool.

COMPONENT 3 · TOOLS

MCP: borrow someone else's toolset

Register an external server in `mcp.json`. The agent gets every tool that server exposes — issue trackers, GitHub PRs, internal APIs — without you writing the integration.

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [ "-y", "@modelcontextprotocol/server-filesystem",
        "${workspaceFolder}/specs",
        "${workspaceFolder}/knowledge" ]
    },
    "linear": {
      "url": "https://mcp.linear.app/mcp",
      "transport": "http"
    }
  }
}
```

Cost: every MCP server adds tools the model has to read past on every turn. Keep the server list short — it's not free context.

Now: Breakout 3 - 12 min

Define one tool for your agent. JSON spec + 3-sentence description. Pick function / script / MCP.

Lab sheet →

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Script-as-tool: JSON declares the interface, the script does the work. The agent reads stdout. This is how you wrap existing CLI tools into the harness without writing new code — your `validate.sh` becomes a tool by adding a JSON descriptor next to it. Minimal commitment, big leverage. Use this when the capability already exists in your team's tooling.

COMPONENT 4 · RULES

Rules: the model's hard stops

Rules are predicates on the agent's actions. **Soft rules** live in the system prompt — the model may forget them. **Hard rules** live in `settings.json` allow/deny lists and hooks — the harness enforces them.

Weak

Vague, unenforced

"Always be helpful and safe.
Don't do anything destructive."

Model may comply most of the time. Will eventually drift. No backstop.

Strong

Specific, enforced

R-20: A spec with status: approved
or implemented is locked. Use
create_spec --revise <id> instead.

(Backed by hooks/pre-tool-use.sh
which blocks the Edit tool call.)

Numbered, specific, paired with a hook. The model can't quietly violate it.

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

MCP is how you mount someone else's tools. Register a server in `mcp.json` — the agent gets every tool that server exposes. Issue trackers, GitHub PRs, internal gateways — no integration code from you. Designer-relevant servers: GitHub, browser automation, ticketing, your design-system API. Caution: too many MCP servers make the agent dumb — too many tools to choose from. Start with one, add only when needed. The 'kitchen sink' is the failure mode here.

COMPONENT 4 · HOOKS

Hooks: deterministic gates around tool calls

Hooks run code at fixed points in the agent's loop. They're how you make a "never" rule actually never. The model can forget; the hook can't.

```
flowchart LR
    M[Model decides]
    to call a tool --> Pre[pre-tool-use.sh]
    Pre -->|exit 0| Tool[Tool runs]
    Pre -->|exit 2 + reason| Block[Blocked]
    reason --> Model
    Tool --> Post[post-tool-use.sh]
    Post --> Next[Next turn]
    style Pre fill:#fef2f2,stroke:#ef4444
    style Post fill:#f0fdf4,stroke:#22c55e
```

Common patterns: pre-tool blocks writes outside scope · post-tool stamps metadata · session-start loads context · stop runs lint.

New · Breakout 4 · 12 min

Write 3 numbered rules (if not already in AGENTS.md), then implement `hooks/pre-tool-use.sh` that blocks your top "never." Pair soft wording + hard gate.

Lab sheet →

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Hooks: deterministic gates around tool calls. PreToolUse fires BEFORE — you can block it. PostToolUse fires AFTER — you can react. The model can forget its rules. The hook can't. Use a hook when 'every time' matters: every time the agent writes a file, every time a tool returns an error, every time the session starts. Hooks > hoping. Designers don't usually write the shell — but you decide WHICH rules deserve a hook vs. a soft prompt.

COMPONENT 5 - KNOWLEDGE

Knowledge base: the domain context the agent always reads

Files in knowledge/ are referenced by the system prompt and loaded on demand by skills. They're how you keep the agent's domain awareness fresh without bloating the prompt.

goes

Stable domain truth

- Schema / data shape definitions
- Workflow diagrams and state machines
- Naming conventions and dictionaries
- Style guides and house rules

stays

Volatile, session-specific

- What the user just said this turn
- Per-session working state → memory
- Secrets, tokens, PII
- Anything that changes daily

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Knowledge base = the domain context the agent reads on demand. Files in knowledge/ are referenced by the system prompt and loaded by skills. Stable info — schemas, dictionaries, workflows, design system facts — goes here. Volatile info — current session, secrets, what the user just said — does NOT. Breakout 6: capture one domain file your agent always needs. 12 minutes — schema, dictionary, or workflow, whichever unblocks more behavior.

COMPONENT 5 - MEMORY

Memory: what the agent remembers across turns

Memory is the working scratchpad — different from the knowledge base (which is stable) and the system prompt (which is identity). It's how the agent keeps state between tool calls without re-asking.

essio

Within one conversation

Notes the agent writes to `notes/SESSION.md` as it works. Cleared on session end.

GOOD FOR

- Tracking partial progress
- Cross-turn working state
- "Last time you said X"

rsiste

Across sessions

Files the agent reads on every `SessionStart` hook. Survives restarts.

GOOD FOR

- User preferences
- Long-running project state
- Decisions made days ago

New - Breakout 5 - 12 min

Add one `knowledge/<topic>.md` your agent should always trust. Document session scratch (`notes/SESSION.md`) vs persistent memory (loaded on `SessionStart`) — where each gets written (often `post-tool-use.sh`).

Lab sheet →

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Memory is the working scratchpad. Different from the knowledge base (stable) and the system prompt (identity). Two types. SESSION — notes/SESSION.md cleared at session end. PERSISTENT — files reread on every SessionStart hook, survive restarts. The post-tool-use hook is often where memory gets WRITTEN. Memory is not a breakout — it's a concept to keep in your head while you build. The hook IS often where memory lives.

WORKFLOWS · SKILL CHAINS

Skill chains & long-running workflows

A chain is not magic — it's **staged prompts** where each stage loads different skills, calls tools, hits hooks, and loops until a guard fires. Rules say what must never happen; hooks enforce it between stages.

```
flowchart TD
    subgraph LOOP["Outer loop (multi-hour)"]
        A["Stage A · Intent skill"] --> B["Stage B · Gather tools + knowledge"]
        B --> C["Stage C · Act · script/MCP tools"]
        C --> D["Hook OK?"]
        D -->|block + reason| A
        D -->|pass| E["Stage D · Verify skill"]
        E --> F["Done / escalate?"]
        F -->|iterate| B
        F -->|ship| G["Persist memory + artifacts"]
    end
    style D fill:#fef2f2,stroke:#f4444
    style F fill:#fff7ed,stroke:#f59e0b
```

Chains compose everything you built today: **meta prompt** stays fixed while **templates** in skills swap per stage; **tools** move data; **hooks** reset or block bad transitions; **loops** close when eval criteria match.

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Two prompt shapes you'll see everywhere in the harness. META = persistent — the agent's frame. Lives in AGENTS.md / CLAUDE.md / system prompt. Loaded once per session, prepended every turn. Sets identity, scope, hard rules, tool roster. TEMPLATE = parameterized — the agent's task slot. Lives in skill bodies, slash commands, tool inputs, hook outputs. Filled in at runtime with task-specific values. Renders per-turn from a fixed shape. Drill the room: every harness component you build is one or the other — or both layered. The system prompt is meta. A skill body is template. A tool description is template. A hook injection is template referenced from a meta. Get them to call out which is which from their own breakout work.

BONUS · CONTEXT ENGINEERING

Meta vs template prompts

Two shapes that show up everywhere in the harness. **Meta** defines the agent. **Template** defines the turn. Most harness components are one or the other — or both, layered.

META PROMPT

Persistent · the agent's frame

- Lives in AGENTS.ad / CLAUDE.ad / system prompt
- Loaded once per session, prepended every turn
- Sets identity, scope, hard rules, tool roster

```
# AGENTS.ad
You are a UX design reviewer.

# RULES
1. Always cite the design token used.
2. Never invent component names.
3. Refer to the system index.

# TASKS
read_file, search_tokens,
validate_spec

# KNOWLEDGE
knowledge/design-system-index.ad
```

TEMPLATE PROMPT

Parameterized · the agent's task slot

- Lives in skill bodies, slash commands, tool inputs, hook outputs
- Filled in at runtime with task-specific values
- Renders per-turn from a fixed shape

```
# skills/design-review.ad (body)
Review the component for
design-token violations.

# INPUT
component_path: {{component_path}}
scope: {{scope}}

# OUTPUT
List of (token, line, fix)

Stop when violations covered
or 5 minutes elapse.
```

Meta defines the agent. Template defines the turn. The system prompt is meta. A skill body is template. A tool description is template. A hook injection is template — referenced from a meta. They layer.

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

This is how subagents and workflows work — without spending time on mechanics. A subagent is just a stage with its own meta + template, returning a result. A workflow is the chain. Each stage's output becomes the next stage's input. The black band across the top is the META — same identity and rules persist across all three stages. The three stage cards are TEMPLATES — each has its own input variables and produces a typed output. The chain isn't code, it's prompts. The harness's job is to route outputs to inputs. When someone asks 'how do I build a multi-step workflow?'; point at this slide. You write three small templates, you let the meta carry across. We won't write orchestration today — but you'll recognize this shape when you read RooCode workflows or LangGraph chains.

BONUS · REFERENCE

Prompt structures across the harness

Every harness component is a shaped prompt. Same four ingredients — identity, rules, inputs, outputs — different surface, different lifecycle.

META Agent

claude-agent / agents.md

```
You are a (person).

RULES
1. (rule)
2. (rule)

TOOLS
(tool list)

PREPROMPT
(Please to load)
```

TEMPLATE Skill

claude/skills/*.md

```
name: design-review
description: when user
asks for a UI review
trigger: /review

1. Load designsystem-
index.md
2. For each section,
run (check)
3. Summarize
(issue, fix, severity)
```

TEMPLATE Tool

tools/*.json

```
{
  "name": "validate_spec",
  "description":
    "We validate on
    a spec.

    - user
      asks 'validate'
    - check
      if [ ] for
      unmerged drafts ",
  "tags": [
    "spec_id", "testing"
  ]
}
```

CONTEXT Hook

claude/hooks/*.sh

```
#!/bin/sh pre-tool-use
tool_id="write_file"

function run_hook {
  Before writing, you
  MUST:
  1. Confirm path to
  be /spec
  2. Include Front-
  matter
  3. Cite the spec ID
}
```

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Reference card. Every component is a shaped prompt. Same four ingredients — identity, rules, inputs, outputs — different surface, different lifecycle. AGENT (META, navy tag) is the long-lived shape in CLAUDE.md / AGENTS.md. SKILL (TEMPLATE, blue tag) is loaded on trigger from .claude/skills/*.md. TOOL (TEMPLATE, blue tag) is invoked when the agent picks it from tools/*.json. HOOK (CONTEXT, amber tag) injects fresh content before/after a step from .claude/hooks/*.sh. Use this slide as a mental cheat-sheet during the breakouts. When someone asks 'where does this rule go?' — the answer is on this card. Pin it open in a second tab if you can.

DAY 1 · CLOSE

Show, tell, sleep on it

Each group: 2 minutes. The single thing you're least sure about — that's tomorrow's first conversation.

SHOW & TELL — 2 MIN PER GROUP

- Which app + which intent slice you're harnessing
- Your file tree on screen
- The one design decision you fought about most
- What you couldn't decide — bring it back tomorrow

HOMEWORK BEFORE DAY 2

- Push your harness / folder to a branch
- Run the agent at home — note 3 things it gets wrong
- Save 2 transcripts where the harness misbehaved
- Bring 1 question about hooks, MCP, or subagents

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

DAY 1 CLOSE. 2 minutes per group. Show the file tree, name the design decision they fought about most, name what they couldn't decide. The undecided question becomes the first conversation tomorrow morning. Homework: push the branch, run the agent at home, save 2 transcripts where the harness misbehaves, bring 1 question.

D2

DAY 2 - 120 MIN

Refine · Test · Demo

Yesterday you built the bones. Today you wire it into a real editor, watch it fail, and harden it.

2 HOURS

Same harness folder, new loop: reproduce a failure, read the trace, change one lever in files or hooks, re-run. **Debugging is the design craft today.**

Wire-up lab · Eval mindset · Demo prep

ALBERT HARRIS ENGINEERING WORKSHOP

SPEAKER NOTES

DAY 2 OPENER. 'Yesterday you built the bones. Today you wire it into a real editor, watch it fail, and harden it.'

Quick recap — anyone bring back a transcript that surprised them? Get 1-2 stories from the room before moving on.

04

SECTION · DAY 2

Agentic UX Patterns

From user flows to intent-system maps.

15 MINUTES

Morning recap in the rear-view. Now we retool the designer lens — how intents, hooks, and states show up when the agent is the surface. **New verbs; same rigor.**

Intent maps · Hooks lifecycle · Subagents & MCP

AGENTIC MARKETS ENGINEERING WORKSHOP

SPEAKER NOTES

Section divider — Agentic UX Patterns. Bridge from the morning recap to the lens shift designers need.

AGENTIC UX		
How our work is changing		
User journey mapping	→	Intent-system mapping How user intents translate into agent capabilities across platforms.
Static wireframes	→	Dynamic orchestration rules Rules for how agents coordinate and interfaces adapt.
Usability testing	→	Real system testing Testing actual multi-agent systems, measuring intent resolution.
Visual design first	→	State modeling first 90% of agentic design work is user-needs modeling and workflows.
Feature requests	→	Capability boundaries What problem does it solve? Agents don't need their own screen.
AGENTIC HARNESS ENGINEERING WORKSHOP		

SPEAKER NOTES

Journey mapping → Intent-system mapping. Wireframes → Orchestration rules. Usability testing → Real system testing. Visual design → State modeling. Feature requests → Capability boundaries. The verb of design has changed. Read the row, give one example each.

AGENTIC UX

Agent time: designing input architecture

Designers influence what the agent knows across three temporal dimensions.



PAST

What the system has already captured

Preferences & goals

Prior decisions

Conversation history

Learned patterns



PRESENT

What is happening right now

Current state

Active constraints

Real-time signals

User context



FUTURE

How the agent adapts based on inputs

Predicted needs

Proactive suggestions

Continual learning

Harness optimization

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Past / Present / Future of agent input architecture. PAST: prefs, history — what should it remember and for how long? PRESENT: current state, signals. FUTURE: proactive suggestions — when helpful, when intrusive? Designers influence all three.

DAY 2 · DETERMINISM

Hooks: the lifecycle you can wire into

AGENTS.md is advisory. Hooks are deterministic. Use them when "every time" matters.

SessionStart
Boot

Inject project state, freshness checks, env probes.

UserPromptSubmit
On user input

Re-read secrets, expand shortcuts, log intent.

PreToolUse
Before action

Block writes outside scope - require confirm for destructive ops.

PostToolUse
After action

Auto-format - run tests - summarize diff.

Stop / SessionEnd
Wrap-up

Persist memory - emit a session report - archive transcripts.

DO

Use a hook when...

Behavior must happen **every time**, not "if it remembers"

Failure has a hard cost (broken build, leaked secret, lost data)

The check is mechanical (regex, schema, file-path scope)

You want a human gate on a destructive action

DON'T

Skip the hook when...

The check needs nuance — keep it in the prompt

It only matters in 1-of-100 turns — too noisy

The model's verdict is the actual logic — let it decide

You'd be flagging style preferences, not safety

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

HOOKS DEEP DIVE. Lifecycle: SessionStart, UserPromptSubmit, PreToolUse, PostToolUse, Stop / SessionEnd. Use a hook when: the rule must hold every time, the failure has hard cost, the check is mechanical, you want a human gate. DON'T use a hook when: the check needs nuance, only matters 1 in 100 turns, the model's verdict IS the logic, or you're flagging style.

DAY 2 - COMPOSITION

Subagents & MCP — when to split, when to plug in



Subagents

Isolated context. Different system prompt. Returns a result, not a transcript.

USE WHEN

- The work has a clean input → output contract (research, review, summarize)
- You want different rules / tools than the parent agent
- You're protecting the main context window from clutter
- You can describe the success criterion in one sentence

FILE

```
.github/agents/researcher.md
---
name: researcher
description: Read-only research; returns a brief.
tools: [webSearch, webFetch, Read]
---
You are a research agent. Be terse. Cite sources.
```



MCP — Model Context Protocol

External tool servers your harness mounts. GitHub, Linear, Slack, browser automation, your own.

DESIGNER-RELEVANT SERVERS

- **GitHub MCP** — issues, PRs, file contents
- **Browser MCP** — let the agent see the rendered UI
- **Filesystem MCP** — bounded read/write for specs and knowledge
- **Custom MCP** — wrap your design-system API

CONFIG

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": [
        "-y",
        "@github/mcp"
      ],
      "linux": {
        "url": "https://mcp.linear.app/mcp",
        "transport": "http"
      }
    }
  }
}
```

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

SUBAGENTS & MCP. Subagents = isolated context, different rules, returns a result. Use when work has a clean input→output contract. MCP = external tool servers your harness mounts. Designer-relevant servers: GitHub, Browser, ticketing, your design-system API. The minute they grok MCP, the question 'how does this reach my issue tracker?' has an answer.

DAY 2 - IMPROVEMENT

Evals are the training data for your harness

You don't fine-tune the model. You fine-tune the harness — by running it on captured tasks and reading the transcripts.

Capture — save real failure transcripts

→

A folder of evals/*.json with input + expected behavior

Replay — re-run them after every harness change

→

CI script that runs the agent against the eval set

Score — pass / fail / regression / needs-review

→

LLM-as-judge or simple assertion checks

Hill-climb — change one thing, measure the delta

→

Tighten prompts, add a hook, narrow tools, retry

Hold the line — don't ship if the eval set regresses

→

Treat the eval set like your design-system regression suite

For designers. Each transcript is a usability test. Same instinct, new artifact — read 5 sessions before you change a tool description.

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

EVALS & HILL-CLIMBING. Capture failures → save as evals → replay after every harness change → score → keep what improves the set. For designers: each transcript is a usability test. Read 5 sessions before changing one tool description.

DAY 2 - WORKING SESSION

Wire your harness into GitHub Copilot or Cline / RooCode

25 minutes. Same groups as Day 1. Open the editor you already use and run your harness against a real task.

1

Drop your harness/ folder into a fresh project
Copilot reads `AGENTS.ad` and `github/copilot-instructions.ad`. RooCode / Cline read `AGENTS.ad` and `roo/rules/`. Move/symlink as needed.

2

Run a real task — the one you reverse-engineered
Drop a real artifact in the workspace and ask the agent to handle it. Harvey: a sample contract — flag risks. Claude Financial Services: a small workbook or diligence slide outline — trace numbers, cite sources, flag what needs human sign-off. Granola: a transcript — produce the brief. Same intent as the live product, your harness.

3

Watch where it diverges from the original product
Tool descriptions too vague? Hook too aggressive? System prompt missing a constraint? Take notes — don't fix yet.

4

Make ONE high-leverage change, re-run
Edit a tool description, tighten the system prompt, or add one hook. Compare before / after.

WHAT TO CAPTURE FOR THE DEMO

before.txt	The transcript before your change.
after.txt	The transcript after your one change.
diff.ad	What you changed and why - 3 bullets max.
tradeoff	What got worse so something else could get better.

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

LIVE WIRE-UP — 25 minutes. Same groups as Day 1. Reminder: the editor IS the runtime — RooCode and Copilot already have the API key, your harness configures HOW they behave. Drop the harness/ folder into a workspace, open it in VS Code, and let the extension's AI play the agent. Run a real task — preferably the one they reverse-engineered. The frontend prototype they built in Breakout 3 should be what they click while the agent works. Watch where the behavior diverges from the original product. Make ONE high-leverage change. Re-run. Capture before.txt, after.txt, diff.md, and the tradeoff. They'll demo this in the next session.

DEMO

DAY 2 · DEMO SESSION

Show your harness

5 minutes per group. Run the agent live, walk through the file tree, defend one tradeoff.

25 MINUTES

This is where it lands. You picked an app, planned, designed, developed, wired it up, watched it fail, and changed one thing. **Now show your work.**

Same groups · 5 min each · 1 min Q&A · Run live in your editor

AGENTIC HARNES ENGINEERING WORKSHOP

SPEAKER NOTES

DEMO SESSION DIVIDER. 5 minutes per group, 1 minute Q&A each. Live in their editor. Set the tone: 'We're not judging finish — we're judging the tradeoff you can defend.' Advance once — the next slide is the two checklists: demo flow and what we're listening for.

DEMO

DAY 2 - DEMO SESSION

Flow & listening

Keep this slide up during demos — presenters follow the checklist; everyone else listens for the signals on the right.

5-MINUTE DEMO FLOW

- ✓ Which app - which intent - which failure mode
- ✓ Show the file tree (15 sec)
- ✓ Run one task live in your editor
- ✓ Show the before/after diff from your one change
- ✓ Name the tradeoff you accepted

WHAT WE'RE LISTENING FOR

- 1 Tool descriptions written like UX copy
- 2 Hooks doing the work that prompts can't be trusted with
- 3 A small, sharp tool list — not the kitchen sink
- 4 An honest failure mode you didn't fully solve
- 5 A tradeoff you can defend in one sentence

ALBERTO BARRETT ENGINEERING WORKSHOP

SPEAKER NOTES

DEMO CHECKLISTS. Left card: 5-minute flow — app + intent + failure mode, file tree, live task, before/after diff, name the tradeoff. Right card: listening criteria — tool copy as UX, hooks doing real work, tight tool list, honest failure mode, defensible tradeoff. Leave this up while groups present.

DAY 2 : DEMO

How we'll judge the demos

Same rubric for every group. Score yourselves first, then we score each other.

CATEGORY	WHAT "GOOD" LOOKS LIKE	PTS
Intent fidelity	The harness handles the chosen intent end-to-end without you having to coach the agent through it.	/ 5
Tool design	Few tools - sharp descriptions written like UX copy - clear permission boundaries.	/ 5
Determinism	At least one hook is doing real work — not relying on the model to remember.	/ 5
Failure design	Group can name the failure mode they targeted and show the before/after of their one change.	/ 5
Tradeoff clarity	One sentence: what you accepted as worse so something else could be better.	/ 5

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

DEMO RUBRIC — score table only. Five categories, 5 points each. Score yourselves first, then we score each other. Next slide: best practices on the left, anti-patterns on the right — leave it up during debrief.

DAY 2 · DEMO

Best practices & anti-patterns

Reference during demos and debrief — what we reward vs. what we'll name in discussion.

BEST PRACTICES WE WANT TO SEE

01

Start with user intent · design the failure state first

02

Cap tools at ~7 · description = UX copy for the model

03

Hooks for "every time"; prompts for "use judgment"

04

One file does one job · AGENTS.md stays under 200 lines

05

Capture transcripts as evals · hill-climb from real failures

ANTI-PATTERNS WE'LL CALL OUT

→

"And here are 23 MCP servers it can use" — too many tools

→

The system prompt is doing the hook's job

→

No way to tell when the agent succeeded vs. gave up

→

Same harness for every user — no scoping at all

→

Demo runs only on the rehearsed task

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

DEMO DO'S AND DON'TS. Left column: best practices — intent-first, cap tools at ~7, hooks for every-time rules, small files, evals. Right column: anti-patterns we call out in discussion — MCP sprawl, prompt doing hook work, no success signal, no user scoping, rehearsed-only demos.

TAKEAWAYS

Your harness engineering toolkit

01

Agent = Model + Harness.

You're always designing the harness, even if you don't call it that.

02

The harness is mostly files.

Markdown for instructions - JSON for config - scripts for hooks. Same pattern across every product.

03

Write tool descriptions like UX copy.

They're the interface between the agent and its capabilities — the highest-leverage micro-copy you'll write all year.

04

Hooks > hoping the model remembers.

Anything that should happen "every time" lives in code, not in a prompt.

05

Most agent failures are input failures.

Read 5 transcripts before you change a single line of the system prompt.

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

TAKEAWAYS. 01 Agent = Model + Harness. 02 The harness is mostly files. 03 Tool descriptions = UX copy. 04 Hooks > hoping. 05 Most agent failures are input failures. Thank the room. Keep the energy up — they earned it.

RESOURCES

Read these next

Source material first, then the docs for the editor you already use. Most of you are on Copilot or RooCode — those rows are the priority.

FOUNDATIONAL ESSAY

The Anatomy of an Agent Harness
Vitaly Tsvetkov's framing of harness components — "If you're not the model, you're the harness."
blog.langchain.dev/the-anatomy-of-an-agent-harness

RESEARCH PAPER

Agentic Harness for Real-World Compilers
[arXiv:2603.20075](https://arxiv.org/abs/2603.20075) — Item audited as the first agentic harness for compiler bugs. Worked anatomy example
arxiv.org/abs/2603.20075

PROTOCOL

Model Context Protocol (MCP)
How to expose external tools — GitHub: Linux, your design-system API — to any harness.
modelcontextprotocol.io

COPILLOT - YOU

Custom instructions in VS Code
Repository: github.com/microsoft/capilot-custom-instructions (plus path-scoped `*.instructions.md` with `applyTo`). Both are read by Copilot on every turn.
code.visualstudio.com/docs/copilot/customization/custom-instructions

COPILLOT - YOU

Repository custom instructions (GitHub Docs)
The official reference for [github.com/capilot-custom-instructions](https://github.com/microsoft/capilot-custom-instructions), including how to compose with personal and org instructions.
docs.github.com/copilot/customizing-copilot/editing-custom-instructions-for-github-copilot

COPILLOT - YOU

Prompt files (.prompt.md)
Reusable, path-modifiable prompts in [github.com/capilot-custom-instructions](https://github.com/microsoft/capilot-custom-instructions). Treat each one like a slash command; participants invoke from Copilot Chat.
docs.github.com/en/copilot/tutorials/customization-library/prompt-files

COPILLOT - YOU

Custom agents (cloud + CLI)
Define agent profiles in [github.com/agents/roam](https://github.com/microsoft/capilot-custom-instructions).ad with YAML front-matter. Copilot's visiting agent loads them on demand.
docs.github.com/en/copilot/concepts/agents/cloud-agent/about-custom-agents

COPILLOT - YOU

Customize AI in VS Code (overview)
Top-level map of every Copilot customization surface: instructions, prompt files, chat modes, MCP, model selection.
code.visualstudio.com/docs/copilot/customization/overview

ROOCODE - YOU

Custom modes
Workspace modes via `roamcode` and global modes in `settings/custom_modes.yaml`.
docs.roamcode.com/features/custom-modes

ROOCODE - YOU

Custom instructions & rules
How `roam/rules/` and `roam/rules-model` compose, plus user-global `roam/rules/`.
docs.roamcode.com/features/custom-instructions

ROOCODE - YOU

Prompt structure (advanced)
The order in which RooCode assembles the system prompt — useful when you're fighting precedence between your files.
docs.roamcode.com/advanced-usage/prompt-structure

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

RESOURCES. Three rows of links. TOP ROW — source material: LangChain's anatomy essay, arXiv 2603.20075, and the MCP spec. SECOND ROW (Copilot — orange stripe) is the priority for most of this room: VS Code custom instructions, the GitHub Docs reference for repo instructions, prompt files, custom agents, and the Customize-AI overview. Walk the room through these explicitly — these are the docs they'll re-open Monday morning. THIRD ROW — RooCode and Cline docs for the rest of the audience. Closing line: 'Keep editing your harness in the open. Push to a shared branch. The harness is a living artifact, not a deliverable.'